

LayerBlade

Proffie – LayerBlade · SD blade styles for ProffieOS

Written for saber builders who change looks in text files instead of recompiling firmware for every color or effect.

October 2026

Contents

Each entry links to that section in this PDF.

Getting started

[What this guide is for](#)

[When you reflash, and when the INI is enough](#)

[INI only \(no new firmware\)](#)

[Firmware upload](#)

[Files to copy](#)

[How a preset picks a look](#)

Building a look

[How layers stack](#)

[Placeholders](#)

[Ignition, retraction, and sound length](#)

[What the firmware does with that number](#)

[Two wipe shapes](#)

[Which lines need extend and retract](#)

[How pixels combine](#)

[Opacity](#)

[What each blend looks like](#)

[Colors and a few numbers](#)

Style catalog

[Base layer styles](#)

[Config bases](#)

[Full named blades](#)

[Classic blades](#)

[Fett263-style full blades](#)

[Texture and effect layers](#)

[Scrolling masks](#)

[Hue waves](#)

[Brightness and color sheets](#)

[Fett263 idle textures](#)

[Events and overlays](#)

[Preon and postoff](#)

[Strip column animations](#)

[The picture file](#)

[Arguments](#)

Examples and reference

[Worked example: preset 0, Strip Column Demo](#)

[A smaller recipe you can edit by hand](#)

[Other named styles in the example presets](#)

[Limits worth knowing](#)

[Where the examples live](#)

Appendix: Named colors

[Section A — SD and layer tokens](#)

[Section B — Fett263 Edit Mode color list](#)

[Section C — Editor catalog extras](#)

Getting started

How to turn the add-on on, which files to copy, and how a preset points at a recipe.

What this guide is for

This guide is for saber builders who change blade looks from files on the SD card. You define the blade, pick a recipe, and stack effects in plain text. You upload new firmware only when the board, the blade count, or the style *names* in the firmware change.

The system is an **opt-in add-on in this fork**. Turn it on in the firmware config you flash (`ENABLE_SD_CONFIG_FILES`). The example profile `config/config-files-config.h` already does that. A ProffieOS build that was compiled without this add-on does not read `blade_styles.ini` . Copying these files onto a card running that kind of image leaves the saber on its compiled presets.

Two files do the everyday work:

File on the SD card	What you edit
<code>config/presets.ini</code>	The preset list: font, track, display name, and one style line per blade.
<code>config/blade_styles.ini</code>	Named recipes. Each <code>[section]</code> is a stack of <code>layer =</code> lines.

A preset points at a recipe with:

```
style = config demo_strip_column
```

`demo_strip_column` is the name inside square brackets in `blade_styles.ini` . Names are case-sensitive.

After you edit either file, put the card back and restart the saber. Color, speed, opacity, and layer changes in these files do not need a new `.bin` upload.

When you reflash, and when the INI is enough

INI only (no new firmware)

- Recolor a recipe, change scroll speed, opacity, or blend mode.
- Add or remove `layer =` lines inside an existing style name the firmware already knows.
- Point a preset at a different `[section]` , or override colors with `key=value` on the style line.
- Swap font, track, or preset name.
- Replace a `.bmp` at the same SD path the recipe already names (same layout and `source_height`).

Firmware upload

- The image you are running was built without `ENABLE_SD_CONFIG_FILES` . Until you flash a build that includes it, the `config/` folder is ignored and `style = config ...` cannot load.

- You change `NUM_BLADES`, pins, LED count, or power wiring. That still lives in the compiled config (and, when you use it, `config/blades.ini` for wiring the example SD blade file already supports).
- You want a style *name* that the flashed image does not contain. New C++ styles are compiled in. After a flash, colors and overlays for that style go back to being INI edits.
- You raise the built-in caps (16 layers, line length). Those are compile-time constants.

Check what the running image actually accepts: open the serial monitor and run `list_named_styles`. If a name is missing there, a `layer =` line that uses it will not parse.

The add-on keeps the rest of Proffie small on purpose. Hardware and the style engine stay in the firmware. The SD files only supply preset strings and layer recipes. Fett263 Edit Mode and the on-saber menus rewrite the preset `style =` line in `presets.ini`. They do not rewrite individual `layer =` rows. For on-saber recolors, put the color in a `{{name}}` placeholder and override it on the style line (`style = config with_vars base=magenta`).

Files to copy

On the card, next to `Fonts/` and `tracks/`:

```
config/presets.ini
config/blade_styles.ini
config/blades.ini          (when you use the SD blade map)
animations/cyan-gold-plasma.bmp  (only for the strip-column demo)
```

Start from `examples/config/` in this repo. The folder on the card must be named `config`. Lines that begin with `#` or `;` are comments. Blank lines are fine.

The shipped example assumes **four blades** (`NUM_BLADES 4`):

Blade index	Typical role in <code>presets.ini</code>
0	Main NeoPixel strip. This is where <code>style = config ...</code> belongs.
1	Simple PWM accent, for example <code>accent_pulse 1500</code> .
2	Simple PWM accent, for example <code>accent_sound_on white 13%</code> .
3	Simple PWM accent, for example <code>accent_glow</code> .

One `style =` line per blade, in index order. A single-strip saber should be flashed with `NUM_BLADES 1` and a single `style =` line per preset. Accent styles drive GPIO or PWM outputs. They are documented briefly at the end of this guide; the pixel-blade recipes are the `config` sections.

If `config/presets.ini` is missing or invalid, the saber uses the presets compiled into the firmware. If a `config` section name is missing from `blade_styles.ini`, that blade style fails to load.

How a preset picks a look

Each preset is a block:

```
new_preset
font = LiquidStatic
track = tracks/hum.wav
style = config demo_strip_column
style = accent_pulse 1500
style = accent_sound_on white 13%
style = accent_glow
name = Strip Column Demo
variation = 0
```

The next `new_preset` starts another look. The file ends with a line that says `end`. You can store up to 64 presets.

`font` is the folder name under `Fonts/` (or at the card root, depending on how your card is laid out). The example voice pack lives in `common/` at the card root. Leave `voice =` out to use `/common`.

`style =` has two forms builders use every day:

```
style = standard cyan white 300 800
style = config sine_waves_cyan
style = config with_vars base=magenta clash=yellow
```

The first form is a **named style** baked into the firmware, with its arguments written out in order. The second form loads every `layer =` line from `[sine_waves_cyan]`. The third form loads `[with_vars]` and replaces the placeholders `{{base}}` and `{{clash}}` for that preset only. Overrides win over the defaults written in the section. You can pass up to 16 `key=value` pairs.

You can mix forms in one preset: blade 0 uses a `config` recipe, and the accent blades use `accent_*` names directly. Those accent names do not read `blade_styles.ini`.

Building a look

How layers stack, how ignition timing works, and how blend modes mix pixels.

How layers stack

Open a section in `blade_styles.ini`. Every `layer =` line is one sheet. **The first line is the bottom. Each following line is stacked on top.** The firmware paints one LED at a time, from the bottom sheet upward, and the last sheet is what you see if it is opaque.

```
layer = lb white           ← top (only while lightning-blocking)
layer = real_clash white 49% ← flash while clashing
layer = multiply opacity 55% sine_waves ...
layer = solid_bend cyan -1 -1 ← bottom: the blade itself
```

A transparent sheet leaves the sheets under it alone. Clash, blast, lockup, drag, melt, lightning block, preon, and postoff are transparent until that event happens. A full blade style such as `rainbow` or `strobe` is opaque. If you stack one of those with the default blend, it covers everything under it. Dim it with a blend and an opacity (next section), or keep opaque styles on the bottom.

Up to **16** `layer =` lines are loaded per section. A single line can be about **380 characters**. Longer lines are truncated.

There is a second way to write one layer, `layer.standard.base = cyan` and similar slot lines. The firmware supports that form for `standard`, `fire`, `rainbow`, `strobe`, `cycle`, `unstable`, and `advanced`. The examples in this guide use the single `layer =` line, which works for every style.

Placeholders

Inside a section you can name values, then drop them into layer lines with double curly braces:

```
[with_vars]
base = cyan
clash = white
ext = 300
ret = 800
layer = standard {{base}} {{clash}} {{ext}} {{ret}}
layer = blast white
```

`{{base}}` is replaced with `cyan` before the style is built. A preset can replace it for one look only:

```
style = config with_vars base=magenta
```

That preset gets a magenta blade. Other presets that say `style = config with_vars` keep cyan. The same pattern is how you recolor a complicated stack without touching every line: `style = config smoke_blade base=cyan`.

Shared color sets can live in a `[palette_something]` section and be pulled in with `palette = something`. A section can also pull layers from another file with `include =`, or nest a whole other recipe with `layer = config other_section`. Nesting is how `[nested_config_demo]` adds a strobe on top of `[base_rainbow_blast]`.

Ignition, retraction, and sound length

Many bases take two times, in milliseconds:

- **extend** (often written `ext`) — how long the blade wipes on.
- **retract** (often written `ret`) — how long it wipes off.

```
ext = 300
ret = 800
layer = solid_bend cyan {{ext}} {{ret}}
```

Use -1 when the wipe should match the sound file. Ignition follows the length of the ignition sound. Retraction follows the retraction sound.

```
ext = -1
ret = -1
layer = solid_bend {{base}} {{ext}} {{ret}}
```

You can also type the `-1` straight on the layer line: `layer = solid cyan -1 -1`.

What the firmware does with that number

In plain terms:

- A value of **1 or more** is a fixed time in milliseconds.
- A value **below 1** (the value you type is `-1`) means "use the WAV length" for that event. The fork calls this choice `InOutMsOrWavLen`. You do not write that name in the INI.

Two wipe shapes

Two wipe shapes exist:

Base you write	Wipe shape
<code>solid</code> , <code>standard</code> , <code>rainbow</code> , and most older full blades	Straight timing. The lit edge moves evenly. Same <code>-1</code> means sound length.

<code>solid_bend</code> , <code>standard_bend</code> , <code>strip_column</code> , and the Fett263 full blades (<code>water_flow</code> , <code>darksaber</code> , <code>static_electricity</code> , <code>power_wave</code> , <code>unstable_blades</code> , <code>fallen_order</code> , <code>thunder_loop</code> , <code>responsive_flame</code> , <code>shimmer_blade</code> , <code>rotoscope</code> , <code>pulse_stripes</code> , <code>kinetic_charge</code> , <code>rotating_pulse</code> , <code>trickle_blade</code>)	Curved wipe. Ignition starts fast and eases at the tip. Retraction starts slow and finishes fast. The fork wraps that curve so <code>-1</code> still tracks the WAV (<code>InOutTrBendAuto</code> using the same millisecond-or-WAV rule).
--	---

If a `strip_column` blade retracts and the retraction sound length is missing, the fallback time is 800 ms.

Which lines need extend and retract

Which lines need `{{ext}}` and `{{ret}}`:

Kind of layer	Put ext/ret on that line?
The bottom base (<code>solid</code> , <code>solid_bend</code> , <code>strip_column</code> , a full named blade)	Yes. This line drives ignition and retraction for the stack.
<code>normal</code> or <code>add</code> textures (<code>gradient_layer</code> , stripes used with <code>add</code> , and similar)	No. They are clipped to the pixels the base has already lit, so they follow the base, including <code>-1</code> .
<code>multiply</code> or <code>screen</code> masks that have no timing of their own (<code>sine_waves</code> , <code>audio_layer</code> , <code>fire_mask</code> , <code>pulse_layer</code> , ...)	No. A dark base already hides them during retract.
<code>hue</code> (<code>hue_waves</code>)	No. A dark or unlit base has nothing to rotate, so the shift follows the base wipe.
<code>smoke_flow</code> , and <code>sparktip_layer</code>	Yes, on every such line, and the numbers must match the base. Smoke has its own in/out.

```

ext = -1
ret = -1
layer = solid {{base}} {{ext}} {{ret}}
layer = multiply opacity 73% smoke_flow black white {{ext}} {{ret}}
layer = screen opacity 12% smoke_flow black {{base}} {{ext}} {{ret}}

```

A preset can push sound-sync into every placeholder at once:

```

style = config smoke_blade ext=-1 ret=-1

```

Preon and postoff are separate from this wipe. They play before ignition and after retraction, and their duration follows the preon or postoff sound on their own. See the events section.

How pixels combine

Each layer line may start with a blend name and an opacity. If you leave them off, the blend is **normal** and the opacity is **full**.

```

layer = multiply opacity 55% sine_waves 2400 0 8192 65535 -2000
layer = add opacity 21% swing white 250
layer = screen opacity 12% smoke_flow black cyan {{ext}} {{ret}}
layer = normal opacity 37% gradient_layer blue cyan
layer = blast white

```

The tokens are `normal`, `multiply`, `screen`, `add`, and `hue`. Then the word `opacity`, then the amount, then the style name and its arguments.

Opacity

Opacity is how strongly that sheet is applied.

You write	Strength
100% or 100	Full. The blend is applied completely.
55% or 55	A bit over half. The sheet is mixed back toward "leave the stack alone."
0%	The layer does nothing.
A whole number above 100 with no percent sign, up to 32768	The raw internal scale. 32768 is full. 16384 is about half. Prefer percents unless you are copying an older line.

What each blend looks like

Token	What you see	Typical use
normal	Paint-over. Opaque pixels replace the blade under them. Partial opacity mixes.	A gentle tint (<code>gradient_layer</code> , <code>rainbow_layer</code>) at low opacity. Also the default for <code>clash</code> and <code>blast</code> , which are already transparent until they fire.
multiply	Darkens. White leaves the color under it unchanged. Black turns that pixel off. Gray dims it.	Stripe masks, sine waves, noise, smoke darkening, a BMP mask.
add	Adds light. Colors pile up and clip at white.	Sparks, swing gleam, bright stripes, a strobe that should flash <i>on top of</i> a rainbow instead of replacing it.
screen	Brightens, softer than add. White wins; black leaves the base alone.	Glow and the bright wisps of smoke.
hue	Rotates the hue of the pixel underneath. Brightness stays about the same. White and gray barely move, because they have no hue. Partial opacity mixes the rotated color back toward the original (it can look a little paler).	<code>hue_waves</code> . Use 100% for a clean shift.

`multiply`, `screen`, and `hue` need a visible base under them. If the base layer failed to draw (for example a missing BMP), those blends stay clear so you do not get a gray or red mask floating on an empty blade. A hue offset of 0 leaves the pixel unchanged.

A worked contrast from the shipped file: `[rainbow_strobe]` puts an opaque strobe over a rainbow with `add` and about half opacity, so the rainbow stays visible between flashes. `[fire_blast]` needs no blend on `blast white`, because blast is clear until you fire a blaster bolt.

Colors and a few numbers

Colors can be a **named string** (`cyan`, `masterswordblue`, `purple`), three channels `0,255,255` in the range 0–255, or `#RRGGBB` / `#RGB` hex. Names are case-insensitive. The **SD and layer token** names in [Section A of the appendix](#) (76 catalog swatches from `parse_color_arg_table.generated.h`) are parsed from INI by firmware. [Section B](#) is the on-saber Fett263 picker (saved as `r,g,b`); hex and `r,g,b` always work.

Mask **min** and **max** (on waves, noise, and similar) are brightness from **0** (black) to **65535** (full white). They are not percents. A multiply wave with min `8192` and max `65535` dims the blade in the trough and leaves it alone at the crest.

`hue_waves` uses the same argument positions, but **min_hue** and **max_hue** are hue offsets, not brightness. See the hue waves section.

Scroll speed is not milliseconds. Larger magnitude moves faster. In the shipped sine-wave recipe, a **negative** speed runs toward the tip and a **positive** speed runs toward the emitter. Width, period, and scale are "how wide is one band" in the same family of units: values around 2000–3000 read as distinct bands; very large values (8000 and up) are wide, slow stripes.

Style catalog

Base styles, textures, combat overlays, and BMP strip animations.

Base layer styles

Use one of these as the **first** `layer = line`, or as the whole `style = line` in `presets.ini` when you do not need a stack.

Config bases

These draw the blade and the in/out wipe, and they leave clash, lockup, and blast to layers you add above them. This is the usual bottom of a `blade_styles.ini` recipe.

Style	Arguments	What you see
<code>solid</code>	base color, extend ms, retract ms	Flat color. Straight wipe. Example: <code>solid cyan 300 800</code> .
<code>solid_bend</code>	base color, extend ms, retract ms	Same flat color with the curved OS7-style wipe. Example: <code>solid_bend cyan {{ext}} {{ret}}</code> .
<code>strip_column</code>	SD path, source height, fps, optional <code>frames_y</code> or <code>frames_x</code> , extend ms, retract ms	A 24-bit BMP played along the blade. Curved wipe. Full write-up in the strip-column section. Example: <code>strip_column animations/cyan-gold-plasma.bmp 144 30 {{ext}} {{ret}}</code> .

Full named blades

These are complete looks. Each one already includes its own clash behavior (and usually lockup and blast). You can use them alone in `presets.ini`, or as the bottom of a stack. If you also add a `clash` or `blast` layer on top, you will get a second hit on top of the one built into the style. For a stack you plan to decorate yourself, prefer `solid` or `solid_bend`.

Classic blades

Style	Arguments in order	What you see
<code>standard</code>	base, clash, extend ms, retract ms, optional lockup, optional blast	Solid blade, clash flash, lockup flicker, blast. Straight wipe. <code>standard cyan white 300 800</code> .
<code>standard_bend</code>	same as <code>standard</code>	Same events, curved wipe.
<code>rainbow</code>	extend ms, retract ms, optional clash, optional lockup	Hue cycle along the blade. <code>rainbow 300 800</code> .
<code>fire</code>	warm color, hot color	Rolling heat from the hilt. No separate extend/retract arguments; the flames grow in. <code>fire red yellow</code> .
<code>unstable</code>	warm, warmer, hot, sparks, extend ms, retract ms	Crackling red-style unstable (BrownNoise, strobe). This is a different style from <code>unstable_blades</code> .
<code>strobe</code>	standby, flash, frequency Hz, flash ms, extend ms, retract ms	Hard flashing between two colors. <code>strobe black white 15 1 300 800</code> .
<code>cycle</code>	start, base, flicker, blast, lockup	Slow color cycle with audio flicker, blast, lockup, and clash.

advanced	hilt, middle, tip, on-spark color, on-spark ms, blast, lockup, clash, extend ms, retract ms, spark tip	Gradient plus spark on ignition and a spark at the extending tip.
gradient	hilt, tip, clash, blast, lockup, extend ms, retract ms	Smooth hilt-to-tip gradient with the usual hits built in.
audio	base, flicker, clash, extend ms, retract ms	The whole blade brightens with the hum.
flicker	warm, hot, clash, extend ms, retract ms	Organic brown-noise flicker.
sparktip	base, clash, extend ms, retract ms, spark tip color	Solid blade with a spark at the leading edge while extending.
sparkle_blade	base, sparkle, blast, lockup, clash, extend ms, retract ms	Solid color plus random sparkles, with blast, lockup, and clash built in.
cylon	scan color, clash, extend ms, retract ms	A scanning band (about a quarter of the blade lit).
pulse_blade	off color, on color, pulse ms, extend ms, retract ms	The whole blade breathes between two colors.

Fett263-style full blades

Fett263-style full blades in this fork. Each takes a curved wipe. Argument order is **base, clash, extend ms, retract ms** unless the row says otherwise. Defaults in parentheses are what you get if you omit the color.

Style	Arguments	What you see
water_flow	base, clash, extend, retract	Bands whose speed and direction follow the blade angle. A hard upward swing can reverse the flow.
darksaber	base, clash, extend, retract	Metallic stripes, brown-noise grain, hum flicker, swing gleam toward white. Default base is a gray-silver.
static_electricity	base, clash, extend, retract	Swing builds a charge; clash or lockup lets it go. Default base is deep sky blue.
power_wave	base, clash, extend, retract	Wide stripes scrolling slowly in reverse. Default base silver.
unstable_blades	base, clash, extend, retract	Crackling stripes whose speed wanders. Default base silver. The name <code>unstable</code> is the other, red crackle blade.
fallen_order	base, clash, extend, retract	Wide stripes with a mid-blade band that pulses about every 800 ms.
thunder_loop	base, clash, extend, retract	Looping storm: a boing-style transition, rolling stripes, and a slow random delay. Default base blue.
responsive_flame	base, clash, extend, retract	Fire that responds to blade angle. Default base red.
shimmer_blade	base, clash, extend, retract	Harder swings make the stripes shimmer faster and longer. Default base cyan.
rotoscope	base, clash, extend, retract	Swing acceleration drives an original-trilogy style band. Default base silver.
pulse_stripes	base, clash, extend, retract	Ignition (and alt-sound) surges the stripe width and speed, with a pulsing mid band. Default base blue.
kinetic_charge	base, kinetic color, clash, extend, retract	Clash and lockup charge up a second color; a long swing lets it decay. Default kinetic color is purple.

rotating_pulse	base, clash, extend, retract	Wide stripes whose direction reverses on a saw wave. Default base blue.
trickle_blade	base, clash, extend, retract	A flame-like trickle, angle stripes, and bands that hold after a swing. Default base green.

charging is the charge-indicator style (battery charging), listed so list_named_styles is not a surprise. It is not a blade recipe. builtin selects a style compiled into a ROM preset; SD recipes use config and the names above instead.

Texture-only twins of the Fett263 blades (water_flow_layer, darksaber_layer, and the other *_layer names) are in the next section. They do not include clash or the wipe. Stack them on solid_bend.

Texture and effect layers

These are the sheets you stack **above** a base. Most of them are masks: use multiply so white means "leave my color" and black means "dim it." A few are color you paint on with normal, add, or screen. hue_waves is different: the hue blend rotates the color already on the blade.

Shipped demos that are "one base plus one mask" live in blade_styles.ini as [demo_saw_waves], [demo_smoothstep_bands], [demo_value_noise], [demo_fbm_noise], [demo_moire_mask], [demo_blade_envelope], [demo_sine_waves_swing], [demo_sine_waves], [demo_hue_waves], [demo_random_bands], [demo_pulse_train], [demo_chirp], and [demo_strip_column]. Presets of the same names are in presets.ini, except [demo_hue_waves], which is a recipe you can point a preset at with style = config demo_hue_waves.

Scrolling masks

Style	Arguments	What you see and how to stack it
sine_waves	Up to four waves, then an optional strength. Each wave is period, phase, min, max, speed. Period 0 turns that wave off.	Smooth brightness waves. multiply opacity 67% sine_waves 2400 0 8192 65535 -2000. [sine_waves_cyan] runs four waves in one line.
saw_waves	Same arguments as sine_waves.	Triangle ramps instead of smooth sine.
hue_waves	Same wave slots as sine_waves, but min and max are hue offsets.	Rolls the color around the wheel. Stack with hue, not multiply. Full write-up below.
sine_waves_swing	The sine-wave arguments, then swing scale and twist scale.	A harder swing or twist shortens the wavelength.
smoothstep_bands	period, speed, min, max, edge width	Soft-edged rolling blocks. Period 0 disables the mask.
pulse_train	period, speed, min, max, duty	Hard on/off squares. Duty is the lit fraction: 50% is half on.
chirp	period, speed, min, max, chirp rate	A sine whose spacing tightens along the blade.
random_bands	speed, band color, optional gap color, optional scale	Uneven bands. A black gap is invisible under multiply, so only the colored bands darken.
stripes	width, speed, color1, color2	Soft moving stripes. Multiply to shade, or add to glow.
hard_stripes	width, speed, color1, color2	The same idea with sharp edges.

value_noise	scale, speed, min, max, optional seed	Smooth 1D noise. Scale 0 disables it.
fbm_noise	scale, speed, min, max, strength	A rougher three-octave noise.
moire_mask	period1, period2, speed1, speed2, min, max	Two ramps that beat against each other.
blade_envelope	center, width, min, max, optional speed	A bump along the blade. Center 0% is the hilt, 100% is the tip, 50% is the middle. Speed 0 holds it still.
fire_mask	warm color, hot color	A scrolling heat pattern. multiply over a base reads as smoke or lava.
smoke_flow	dark, light, extend ms, retract ms, optional speed	Wide dual smoke bands. Ext and ret must match the base. Speed 1 is the default roll; 2 is twice as fast. Multiply black white for shade; screen black plus the base color for bright wisps.
smoke_up / smoke_down	dark, light, extend ms, retract ms	Older one-direction smoke. Shipped recipes use smoke_flow instead.
noise_flicker	base color, flicker color	Organic crackle. Multiply over a base.
unstable_stripes	base color	The crackling stripe band from unstable_blades, without the rest of that blade.
strip_column_mask	SD path, source height, fps	A BMP used as a brightness mask. Always the frames_y layout. See the strip-column section.

Hue waves

hue_waves scrolls up to four sine-shaped **hue** offsets along the blade. It does not dim the blade. The pixels under the layer stay the same brightness and move around the color wheel.

Use the **hue** blend. multiply treats the offset as a dark mask and crushes the blade toward black. normal paints that offset as a red level and replaces the color underneath.

```
layer = hue opacity 100% hue_waves 2400 0 0 8192 -2000
```

100% is a clean rotation. A lower opacity mixes the shifted color back toward the original, which can look a little paler. To soften the effect, keep opacity at 100% and use a smaller max_hue, or the optional strength argument.

Put this above a saturated base (cyan , red , blue). White and gray barely change, because rotation needs a hue to begin with.

Argument	What it does
period	Wavelength along the blade, same units as sine_waves . About 2000–3000 is a good start. 0 turns that wave off.
phase	Spatial offset, same units as sine_waves . 0 is the usual start.
min_hue	Hue offset at the trough. 0 leaves the base color alone there.

max_hue	Hue offset at the crest. 8192 is about a quarter turn (90 degrees).
speed	Scroll speed, same sign and scale as sine_waves . Negative runs toward the tip.
strength	Optional last number, after the wave slots. 65535 (the default) is the full offset. 0 is no shift.

Hue numbers are the same units as Proffie's RotateColors / Hue styles:

Value	Turn
0	No shift.
8192	About 90 degrees.
16384	180 degrees (the opposite side of the wheel).
32768	360 degrees, which looks like no shift.

You can pass integers up to 65535 . The wheel repeats every 32768 . A range of 0 to 65535 walks the wheel about twice along one period.

Each wave is five numbers: period phase min_hue max_hue speed . You can repeat that up to four times. Unused waves default to period 0 and add nothing. Active waves **add** their offsets. They do not multiply.

Two waves:

```
layer = hue opacity 100% hue_waves 2400 0 0 8192 -2000 1600 512 0 4096 1400
```

Strength is argument 21, so it comes after all four slots. With one wave, leave it off. To set it, pad the unused slots with period 0 , then the strength:

```
layer = hue opacity 100% hue_waves 2400 0 0 8192 -2000 0 0 0 8192 0 0 0 0 8192 0 0 0 0 8192 0
32768
```

That last 32768 is half strength.

Shipped recipe: [demo_hue_waves] in blade_styles.ini (cyan solid plus the one-wave line above, then the usual clash, blast, and lockup). It is not a numbered preset. Point a preset at it with style = config demo_hue_waves .

Brightness and color sheets

These have no extend/retract arguments of their own (except sparktip_layer).

Style	Arguments	What you see and how to stack it
audio_layer	none	The whole blade brightens with the hum. multiply opacity 100% audio_layer .
pulse_layer	optional pulse ms (default 3000)	A slow global breathe. Multiply.

base_flicker	delta percent, min period ms, max period ms	The whole blade flickers in brightness together.
swing_layer	delta percent, speed threshold	Brighter while you swing, hue kept (unlike add of white, which washes color). Multiply.
per_led_flicker	none	Each LED crackles on its own. Multiply, often around 40%.
gradient_layer	hilt color, tip color	A hilt-to-tip wash. normal plus opacity: 100% replaces the base, about 10% is a tint. Clipped to the base wipe.
rainbow_layer	none	An animated rainbow tint. Same normal plus opacity treatment.
sparktip_layer	spark color, extend ms, retract ms	Spark at the extending tip only. Match ext/ret to the base. Stack with add.
cylon_layer	scan color, on percent, RPM	Scanner band. Stack with add so the black part of the scan adds nothing.
pixel_sequence	a step string	A timed chase. Step format is in doc/pixel_sequencer.md.

Fett263 idle textures

Stack with normal or multiply over solid_bend. They are the moving part of the full blade of the same name, without clash, lockup, blast, or the wipe.

Style	Arguments
water_flow_layer	base color
darksaber_layer	base color
static_electricity_layer	base color
power_wave_layer	base color
fallen_order_layer	base color
shimmer_blade_layer	base color
rotoscope_layer	base color
pulse_stripes_layer	base color
kinetic_charge_layer	base color, kinetic color
rotating_pulse_layer	base color
trickle_blade_layer	base color
thunder_loop_layer	base color
responsive_flame_layer	base color

The SD recipes [water_blade], [darksaber_blade], [static_electricity_blade], [power_wave_blade], [unstable_blades], [fallen_order_blade], [shimmer_blade_sd], [rotoscope_sd], [pulse_stripes_sd], [kinetic_charge_sd], [rotating_pulse_sd], [trickle_blade_sd], [thunder_storm_sd], and [responsive_flame_sd] show these textures with

combat layers already attached. Several of those looks also exist as the single full-blade names in the previous section (`style = water_flow blue white 300 800`), which need the matching firmware but no layer file.

Events and overlays

Add these above the base. Most stay invisible until the saber actually does that thing, so they usually need no blend and no opacity. Where a recipe uses `add`, it is because the effect should brighten the blade instead of painting a flat color.

You want	Layer line	When it shows
Clash flash	<code>clash white</code>	A short full-blade flash on clash.
Clash in one place	<code>localized_clash white</code>	A band at a random spot along the blade.
Clash that follows the angle	<code>responsive_clash white</code>	A bump placed from the blade angle.
Real Clash (OS7)	<code>real_clash white 49%</code>	Strength of the hit picks a bump, wave, spark, or fade. <code>49%</code> pins the band on the blade. <code>angle</code> lets the band follow tilt: <code>real_clash white angle</code> . This is what <code>[demo_strip_column]</code> uses.
Lockup	<code>lockup white</code> or <code>responsive_lockup white</code>	While a lockup is held. The bump follows blade angle. Both names are the same effect.
Blast, fixed wave	<code>blast white</code>	A blast event. Timing is fixed in the firmware (about 200 / 100 / 400 ms). Only the color is yours. <code>[fire_blast]</code> and <code>[smoke_laser]</code> use this.
Blast, random wave	<code>blast_wave_random white</code>	An OS7-style wave whose size and place vary. <code>[demo_strip_column]</code> uses this.
Blast that follows the angle	<code>responsive_blast white</code>	Wave placed from the blade angle. <code>[composable_checklist_responsive]</code> uses this.
Drag	<code>drag orange</code>	Drag lockup. Twist changes the effect.
Melt	<code>melt orange_red</code>	Melt lockup. Twist changes the effect.
Lightning block	<code>lb white</code>	Lightning-block lockup.
Swing gleam	<code>add opacity 21% swing white 250</code>	Brighter while the saber is moving. The number is a speed threshold.
Sparkles	<code>add opacity 15% sparkle white</code>	Random twinkles. Add or screen keeps them from painting a solid sheet.
Idle pulse	<code>pulse white 3000</code>	A breathing color. Use <code>add</code> or <code>multiply</code> .
Ignition flash	<code>ignition_flash white 300 600</code>	A full-blade flash during the extension itself (after preon). Held for the extend time, then fades over the third number.

Force	add opacity 37% force_glow white	While a Force effect plays and the blade is on. Brightness follows that sound. The font needs force sounds, and a button must be mapped to Force.
-------	-------------------------------------	---

Preon and postoff

Preon plays after the button press and before the main blade ignites. **Postoff** plays after the blade has retracted, before the LEDs power down. Duration matches the sound file in the font (`preon/` and `postoff/`). You only pass a color. Glow and sputter also follow how loud that sound is, so they fade as the file fades. They are transparent the rest of the time.

Style	What you see
<code>preon_glow</code>	The whole blade glows. Louder preon audio is brighter. <code>[mystic_awakening]</code> uses blue.
<code>preon_wipe</code>	Color sweeps hilt to tip across the preon sound. <code>[spectral_gate]</code> uses green.
<code>preon_sputter</code>	Lit length follows loudness: loud pushes the blade out, quiet pulls it back. <code>[sputter_gate]</code> .
<code>postoff_glow</code>	Same glow idea after retraction.
<code>postoff_wipe</code>	Color drains tip to hilt across the postoff sound.
<code>postoff_sputter</code>	Length follows the postoff sound, after the main blade is gone.

`[inferno_ritual]` shows the same idea on a fire base: orange `preon_glow`, then fire, then a red `postoff_wipe`.

A typical combat stack, copied from the texture demos:

```
layer = real_clash {{clash}} 49%
layer = blast_wave_random {{clash}}
layer = responsive_lockup {{lockup}}
layer = lb {{lb}}
```

`[smoke_laser]` uses the simpler `clash` and `blast` pair, plus drag, melt, and lightning block.

`[composable_checklist]` is the long example: gradient tint, hum, sparkle, preon, postoff, spark tip, real clash, random blast, lockup, drag, melt, lightning block, and swing.

Strip column animations

`strip_column` is a **base**. It plays a normal Windows BMP from the SD card along the blade, with the curved ignition and retraction wipe. `strip_column_mask` is a **texture**: the same kind of file, turned into a gray mask you multiply over some other base.

The picture file

Export a **24-bit uncompressed BMP** (no RLE, no palette). GIMP: Export As `.bmp`, 24-bit, do not run-length encode. Photoshop: Save As BMP, 24-bit, Windows, uncompressed.

The file is a flipbook. Which way time runs depends on a word in the layer line. **If you omit that word, the layout is `frames_y`.**

Word	How to paint the BMP	Example size
<code>frames_y</code> (default). Aliases: <code>row</code> , <code>rows</code> .	Width is pixels along the blade. Height is the number of frames. The top row is the hilt, the bottom row is the tip. One row is one moment in time.	144 wide by 120 tall means 144 blade pixels and 120 frames.
<code>frames_x</code> . Aliases: <code>column</code> , <code>columns</code> .	Width is the number of frames. Height is pixels along the blade. One vertical column is one moment. The top of the image is the hilt.	27 wide by 144 tall means 27 frames of a 144-pixel blade.

`source_height` in the layer line is the **blade span in pixels** (the blade axis of the file, not the frame count). For `frames_y` that number is the BMP **width**. The shipped demo uses `144`. If you set `source_height` larger than the file's blade span, the firmware clamps it and prints a serial warning. The in-memory column holds at most 170 pixels.

Color on disk is stored BGR; the loader turns it into RGB. The blade resamples the column to however many LEDs you actually have.

The picture is opened **when the saber is on**, not during boot. A missing file or a BMP that is not 24-bit uncompressed logs a line on serial and that layer stays clear (the log text says the layer is transparent). Quote a path that contains spaces: `"animations/my file.bmp"`.

Arguments

```
strip_column <path> <source_height> <fps> [frames_y|frames_x] <extend_ms> <retract_ms>
strip_column_mask <path> <source_height> <fps>
```

`fps` is how fast you *want* the frames to advance. The firmware steps one frame at a time. If the card is slow it will lag rather than skip ahead, so the motion can run slower than the number you wrote. Masks such as `sine_waves` scroll on their own clock and can look faster than the flipbook. To see the BMP alone, comment the multiply line out, or try a low `fps` such as `2`.

Ring buffer and SD reads (firmware)

SD access runs while the blade style loop ticks and must share the card with audio (`LOCK_SD`). To avoid hitching every frame, ProffieOS keeps a **ring of decoded columns** in RAM (default **6** slots, **512 bytes** each — enough for about **170** RGB pixels along the blade). While one frame displays, earlier ticks **prefetch** the next frames into free slots.

- **`frames_y` (default):** When only one frame is buffered ahead, the loader may read up to **four** upcoming BMP **rows** in one seek (bulk path in `strip_column_bmp.h`). If the flipbook loops and those rows are not contiguous in the file, it falls back to loading one frame at a time.
- **`frames_x`:** Each animation tick loads the next vertical column in **slices** (several pixel rows of the column per tick).

Playback still never **skips** frames to catch up — under load you get slower motion, not jumps. Compile-time knobs include `STRIP_COLUMN_FRAME_RING_SIZE`, `STRIP_COLUMN_BMP_Y_BULK_ROWS`, and `STRIP_COLUMN_Y_AHEAD_WATERMARK` (see `strip_column.h` / `strip_column_bmp.h`).

LayerBlade: On `#/styles`, upload a matching 24-bit BMP on a `strip_column` layer to preview the flipbook in the browser (path string must match what you export). This does not replace on-saber SD timing tests.

`strip_column_mask` does not take an extend time or a frame-axis word. It always uses `frames_y`. A file that is one pixel tall is a still mask. Paint gray: under `multiply`, white leaves the blade alone and black darkens it. Colored pixels are averaged to gray. Requires firmware with the `strip_column_mask` named style (see `list_named_styles` over serial). In the shipped `[demo_strip_column]` the mask line is left commented.

```
layer = multiply opacity 80% strip_column_mask masks/plasma_mask.bmp 144 30
```

A still crackle mask is the same with a short file and `fps` of `1`:

```
layer = multiply opacity 80% strip_column_mask masks/crackle.bmp 144 1
```

Examples and reference

A full preset to copy, a smaller recipe, the accent styles used in the examples, and the hard limits.

Worked example: preset 0, Strip Column Demo

Preset 0 in `examples/config/presets.ini` is the pattern to copy.

On the card

1. `config/presets.ini` and `config/blade_styles.ini` from `examples/config/`.
2. The BMP named by the recipe. The layer line loads `animations/cyan-gold-plasma.bmp`. The file in the repo is `examples/sd/animations/cyan-gold-plasma.bmp`. Copy it to that animations path on the card. The section comment also points at `examples/sd/animations/cyan-plasma.bmp` as a 144-by-120 `frames_y` sample (120 frames, 144 pixels along the blade). `source_height` in the recipe is 144, which matches that blade span.
3. A font folder the preset names (`LiquidStatic`) and `tracks/hum.wav`, plus `common/` if you want voice prompts.
4. Firmware flashed with `ENABLE_SD_CONFIG_FILES`, and `NUM_BLADES` matching the four `style =` lines (or edit the preset down to the blades you have).

The preset

```
new_preset
font = LiquidStatic
track = tracks/hum.wav
style = config demo_strip_column
style = accent_pulse 1500
style = accent_sound_on white 13%
style = accent_glow
name = Strip Column Demo
variation = 0
```

Blade 0 loads the section below. Blades 1–3 are PWM accents, not part of the pixel stack.

The recipe (`[demo_strip_column]` in `blade_styles.ini`)

```
[demo_strip_column]
ext = -1
ret = -1
clash = white
lockup = white
lb = white
layer = strip_column animations/cyan-gold-plasma.bmp 144 30 {{ext}} {{ret}}
# layer = multiply opacity 80% strip_column_mask masks/plasma_mask.bmp 144 30
layer = multiply opacity 55% sine_waves 2400 0 8192 65535 -2000
layer = real_clash {{clash}} 49%
layer = blast_wave_random {{clash}}
layer = responsive_lockup {{lockup}}
layer = lb {{lb}}
```

What each line does:

1. `ext = -1` and `ret = -1` store "match the sound" in placeholders.
2. The first layer is the BMP base. Path, blade span `144`, `30` frames per second, then the two times. There is no `frames_y` word, so the default layout is used: width is the blade, height is the frame count. Ignition and retraction use the curved wipe, timed from the WAV because the times are `-1`.
3. The commented line is an optional BMP mask at 80% multiply. It is not active.
4. One sine wave multiplies the picture at 55% strength. Period `2400`, phase `0`, brightness from `8192` to `65535`, speed `-2000` (toward the tip in the shipped wave recipes). The BMP still shows through the bright part of the wave.
5. Clash uses Real Clash with a band centered at 49% along the blade, color taken from `{{clash}}`.
6. Blasts throw a random wave in that same color.
7. Lockup is the angle-following bump.
8. Lightning block uses `{{lb}}`.

To recolor only the hits, either edit `clash =` in the section or, for one preset, write:

```
style = config demo_strip_column clash=yellow
```

To hold a fixed wipe instead of the sound length:

```
style = config demo_strip_column ext=300 ret=800
```

Restart after you save the card.

A smaller recipe you can edit by hand

Flat cyan, curved wipe, sound-synced, with the same combat layers as the demo:

```

[my_cyan]
base = cyan
ext = -1
ret = -1
clash = white
lockup = white
lb = white
layer = solid_bend {{base}} {{ext}} {{ret}}
layer = real_clash {{clash}} 49%
layer = blast_wave_random {{clash}}
layer = responsive_lockup {{lockup}}
layer = lb {{lb}}

```

```

new_preset
font = LiquidStatic
track = tracks/hum.wav
style = config my_cyan
name = My Cyan
variation = 0

new_preset
font = LiquidStatic
track = tracks/hum.wav
style = config my_cyan base=orange
name = My Orange
variation = 0

```

Both presets share the recipe. The second one replaces `{{base}}` with orange. If this saber has accent blades, add the extra `style =` lines after the main blade, the same way preset 0 does.

Other named styles in the example presets

The example `presets.ini` puts these on blades 1–3. They are for simple PWM or GPIO accents. They turn off when the saber is retracted (except `accent_sound_on`, which follows audio level and can stay active through the postoff tail).

Style	Example	Role
<code>accent_pulse</code>	<code>accent_pulse 1500</code>	White pulse. The number is the period in milliseconds.
<code>accent_sound_on</code>	<code>accent_sound_on white 13%</code>	Full on while audio is above the threshold.
<code>accent_glow</code>	<code>accent_glow</code>	Brightness follows the hum.
<code>accent_on</code>	<code>accent_on</code>	Solid on while the blade is out.

Further accent names (`accent_clash`, `accent_lockup`, `accent_strobe`, `accent_battery`, and others) are listed in the header of `examples/config/presets.ini` and in `examples/README.md`.

Limits worth knowing

Limit	Value
Layers in one section	16
Characters on one layer line	384
<code>name = value</code> placeholders in one section	16
<code>key=value</code> overrides on one <code>style = config</code> line	16
Presets in <code>presets.ini</code>	64
Palette blocks cached	8
Nested <code>include</code> depth	4

Section names are case-sensitive. A broken line is skipped; it does not crash the saber, and it does not always tell you on the blade. If a look is missing a layer, check spelling against `list_named_styles` and the square-bracket name in `style = config`.

Very long preset lists with very long `style =` lines use RAM. If a board with a small memory budget misbehaves, shorten the preset list before you shorten the recipes.

Where the examples live

Path	What it is
<code>examples/config/blade_styles.ini</code>	Every shipped <code>[section]</code> , with comments on the layer lines.
<code>examples/config/presets.ini</code>	Preset 0 is Strip Column Demo. Later presets call the other sections.
<code>examples/sd/animations/</code>	Sample BMPs, including <code>cyan-gold-plasma.bmp</code> and <code>cyan-plasma.bmp</code> .
<code>doc/README_blade_styles_config.md</code>	Same system, written for people editing recipes all day, including the number scales.
<code>doc/sd_config.md</code>	How <code>presets.ini</code> replaces the compiled preset list.
<code>examples/README.md</code>	What each file in <code>examples/config/</code> is for, and the accent styles.

Appendix: Named colors

ProffieOS color names fall into **three scopes**, matching the LayerBlade editor catalog:

- **SD and layer tokens** (`styles/parse_color_arg_table.generated.h` , merged catalog + Fett263) — names you can type in `blade_styles.ini` , `layer =` lines, and `{{placeholder}}` overrides. Firmware resolves them with `ParseColorName` in `styles/parse_color_arg.h` . Regenerate the table with `node tools/generate-parse-color-names.js` .
- **Fett263 Edit Mode list** (`props/saber_fett263_buttons.h` `color_list_`) — the on-saber color picker when Fett263 props are enabled. Voice labels come from `ColorNumber` in `sound/sound_library.h` . Choosing a color **rewrites the preset as `r,g,b`** ; many of the same colors also work as text names in Section A after you flash a build with the generated table.
- **Editor catalog extras** (Section C, if any) — names in `colors.json` extended + vivid that are **not** in the firmware table. Use `r,g,b` or `#hex` in INI for those; when Section C is empty, every catalog name is already in Section A.

You can always use `r,g,b` (channels **0–255**) or `#RRGGBB` / `#RGB` hex anywhere a color argument is accepted. Matching for SD names is **case-insensitive**.

Generated **2026-10-04** · Section A: 76 · Section B: 27 · Section C: 0. Re-run `node examples/generate-named-colors-appendix.js` and rebuild the PDFs after firmware or catalog changes.

Section A — SD and layer tokens

Source: `styles/parse_color_arg_table.generated.h` (`ParseColorName`).

Swatch	Name	Rgb (0–255)	Hex	Scope
	red	255, 0, 0	#ff0000	SD name
	green	0, 255, 0	#00ff00	SD name
	blue	0, 0, 255	#0000ff	SD name
	cyan	0, 255, 255	#00ffff	SD name
	yellow	255, 255, 0	#ffff00	SD name
	magenta	255, 0, 255	#ff00ff	SD name
	white	255, 255, 255	#ffffff	SD name
	black	0, 0, 0	#000000	SD name
	orange	255, 128, 0	#ff8000	SD name
	darkorange	255, 68, 0	#ff4400	SD name
	deeppink	255, 0, 75	#ff004b	SD name
	deepskyblue	0, 135, 255	#0087ff	SD name
	dodgerblue	2, 72, 255	#0248ff	SD name

	hotpink	255, 36, 118	#ff2476	SD name
	pink	255, 136, 154	#ff889a	SD name
	tomato	255, 31, 15	#ff1f0f	SD name
	coral	255, 55, 19	#ff3713	SD name
	aqua	0, 255, 255	#00ffff	SD name
	lime	0, 255, 0	#00ff00	SD name
	fuchsia	255, 0, 255	#ff00ff	SD name
	springgreen	0, 255, 55	#00ff37	SD name
	steelblue	14, 57, 118	#0e3976	SD name
	silver	100, 100, 150	#646496	SD name
	greenyellow	108, 255, 6	#6cff06	SD name
	chartreuse	55, 255, 0	#37ff00	SD name
	orangered	255, 14, 0	#ff0e00	SD name
	gold	180, 130, 0	#b48200	SD name
	aquamarine	55, 255, 169	#37ffa9	SD name
	iceblue	30, 60, 200	#1e3cc8	SD name
	indigo	43, 0, 210	#2b00d2	SD name
	purple	93, 0, 197	#5d00c5	SD name
	deeppurple	118, 0, 194	#7600c2	SD name
	glacier	85, 85, 200	#5555c8	SD name
	icewhite	180, 180, 255	#b4b4ff	SD name
	lightcyan	191, 255, 255	#bffffff	SD name
	moccasin	255, 199, 119	#ffc777	SD name
	lemonchiffon	255, 244, 157	#fff49d	SD name
	navajowhite	255, 187, 108	#ffb6c	SD name
	electricpurple	127, 0, 255	#7f00ff	SD name
	electricviolet	71, 0, 255	#4700ff	SD name
	electriclime	156, 255, 0	#9cff00	SD name
	amber	255, 135, 0	#ff8700	SD name
	cyberyellow	255, 168, 0	#ffa800	SD name
	canaryyellow	255, 221, 0	#ffdd00	SD name
	palegreen	28, 255, 28	#1cff1c	SD name
	flamingo	255, 80, 154	#ff509a	SD name

	vividviolet	90, 0, 255	#5a00ff	SD name
	psychedelicpurple	186, 0, 255	#ba00ff	SD name
	hotmagenta	255, 0, 156	#ff009c	SD name
	brutalpink	255, 0, 128	#ff0080	SD name
	neonrose	255, 0, 55	#ff0037	SD name
	vividraspberry	255, 0, 38	#ff0026	SD name
	haltred	255, 0, 19	#ff0013	SD name
	moltencore	255, 24, 0	#ff1800	SD name
	safetyorange	255, 33, 0	#ff2100	SD name
	orangejuice	255, 55, 0	#ff3700	SD name
	imperialyellow	255, 115, 0	#ff7300	SD name
	schoolbus	255, 176, 0	#ffb000	SD name
	supersaiyan	255, 186, 0	#ffba00	SD name
	star	255, 201, 0	#ffc900	SD name
	lemon	255, 237, 0	#ffed00	SD name
	electricbanana	246, 255, 0	#f6ff00	SD name
	busybee	231, 255, 0	#e7ff00	SD name
	zeusbolt	219, 255, 0	#dbff00	SD name
	limezest	186, 255, 0	#baff00	SD name
	limoncello	135, 255, 0	#87ff00	SD name
	cathodegreen	0, 255, 22	#00ff16	SD name
	mintyparadise	0, 255, 128	#00ff80	SD name
	plungepool	0, 255, 156	#00ff9c	SD name
	vibrantmint	0, 255, 201	#00ffc9	SD name
	masterswordblue	0, 255, 219	#00ffdb	SD name
	brainfreeze	0, 219, 255	#00dbff	SD name
	blueribbon	0, 33, 255	#0021ff	SD name
	rareblue	0, 13, 255	#000dff	SD name
	overdueblue	13, 0, 255	#0d00ff	SD name
	violentviolet	55, 0, 255	#3700ff	SD name

Section B — Fett263 Edit Mode color list

Source: `props/saber_fett263_buttons.h` `color_list_` · voice: `sound_library.h` `ColorNumber`
 (`SayColor`).

Swatch	#	Voice label	Rgb (0–255)	Hex	Scope
	1	Red	255, 0, 0	#ff0000	Edit menu only; save as r,g,b on SD
	2	OrangeRed	255, 14, 0	#ff0e00	Edit menu only; save as r,g,b on SD
	3	DarkOrange	255, 68, 0	#ff4400	Edit menu only; save as r,g,b on SD
	4	Orange	255, 97, 0	#ff6100	Edit menu only; save as r,g,b on SD
	5	Gold	180, 130, 0	#b48200	Edit menu only; save as r,g,b on SD
	6	Yellow	255, 255, 0	#ffff00	Edit menu only; save as r,g,b on SD
	7	GreenYellow	108, 255, 6	#6cff06	Edit menu only; save as r,g,b on SD
	8	Green	0, 255, 0	#00ff00	Edit menu only; save as r,g,b on SD
	9	AquaMarine	55, 255, 169	#37ffa9	Edit menu only; save as r,g,b on SD
	10	Cyan	0, 255, 255	#00ffff	Edit menu only; save as r,g,b on SD
	11	DeepSkyBlue	0, 135, 255	#0087ff	Edit menu only; save as r,g,b on SD
	12	DodgerBlue	2, 72, 255	#0248ff	Edit menu only; save as r,g,b on SD
	13	Blue	0, 0, 255	#0000ff	Edit menu only; save as r,g,b on SD
	14	IceBlue	30, 60, 200	#1e3cc8	Edit menu only; save as r,g,b on SD
	15	Indigo	43, 92, 210	#2b5cd2	Edit menu only; save as r,g,b on SD
	16	Purple	93, 0, 197	#5d00c5	Edit menu only; save as r,g,b on SD
	17	DeepPurple	118, 0, 194	#7600c2	Edit menu only; save as r,g,b on SD
	18	Magenta	255, 0, 255	#ff00ff	Edit menu only; save as r,g,b on SD
	19	DeepPink	255, 0, 75	#ff004b	Edit menu only; save as r,g,b on SD
	20	Silver	100, 100, 150	#646496	Edit menu only; save as r,g,b on SD
	21	Glacier	85, 85, 200	#5555c8	Edit menu only; save as r,g,b on SD
	22	IceWhite	180, 180, 255	#b4b4ff	Edit menu only; save as r,g,b on SD
	23	LightCyan	191, 255, 255	#bffffff	Edit menu only; save as r,g,b on SD
	24	Moccasin	255, 199, 119	#ffc777	Edit menu only; save as r,g,b on SD
	25	LemonChiffon	255, 244, 157	#fff49d	Edit menu only; save as r,g,b on SD
	26	NavajoWhite	255, 187, 108	#ffb66c	Edit menu only; save as r,g,b on SD
	27	White	255, 255, 255	#ffffff	Edit menu only; save as r,g,b on SD